

2. Pull out the sqrt call

The first optimization was to pull the sqrt call out of the limit test, just in case the compiler wasn't optimizing that correctly, this one is faster:

```
int is_prime(int n) {
    long lim = (int) sqrt(n);
    for (int i=2; i<=lim; i++) if (n%i == 0) return 0; return 1;}

```

3. Restatement of sqrt.

```
int is_prime(int n) {
    for (int i=2; i*i<=n; i++) if (n%i == 0) return 0;
    return 1;
}

```

4. We don't need to check even numbers

```
int is_prime(int n) {
    if (n == 1) return 0;           // 1 is NOT a prime
    if (n == 2) return 1;          // 2 is a prime
    if (n%2 == 0) return 0;        // NO prime is EVEN, except 2
    for (int i=3; i*i<=n; i+=2)    // start from 3, jump 2 numbers
        if (n%i == 0)             // no need to check even numbers
            return 0;
    return 1;
}

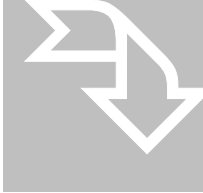
```

5. Other prime properties

A (very) little bit of thought should tell you that no prime can end in 0,2,4,5,6, or 8, leaving only 1,3,7, and 9. It's fast & easy. Memorize this technique. It'll be very helpful for your programming assignments dealing with relatively small prime numbers (16-bit integer 1-32767). This divisibility check (step 1 to 5) will not be suitable for bigger numbers. First prime and the only even prime: 2. Largest prime in 32-bit integer range: $2^{31} - 1 = 2,147,483,647$

6. Divisibility check using smaller primes below sqrt(N):

Actually, we can improve divisibility check for bigger numbers. Further investigation concludes that a number N is a prime if and only if no primes below sqrt(N) can divide N.



How to do this ?

1. Create a large array. How large?
2. Suppose max primes generated will not greater than $2^{31}-1$ (2,147,483,647), maximum 32-bit integer.
3. Since you need smaller primes below $\sqrt{N}$, you only need to store primes from 1 to $\sqrt{2^{31}}$
4. Quick calculation will show that of $\sqrt{2^{31}} = 46340.95$.
5. After some calculation, you'll find out that there will be at most 4792 primes in the range 1 to 46340.95. So you only need about array of size (roughly) 4800 elements.
6. Generate that prime numbers from 1 to 46340.955. This will take time, but when you already have those 4792 primes in hand, you'll be able to use those values to determine whether a bigger number is a prime or not.
7. Now you have 4792 first primes in hand. All you have to do next is to check whether a big number N a prime or not by dividing it with small primes up to $\sqrt{N}$. If you can find at least one small primes can divide N , then N is not prime, otherwise N is prime.

Fermat Little Test:

This is a probabilistic algorithm so you cannot guarantee the possibility of getting correct answer. In a range as big as 1-1000000, Fermat Little Test can be fooled by (only) 255 Carmichael numbers (numbers that can fool Fermat Little Test, see Carmichael numbers above). However, you can do multiple random checks to increase this probability.

Fermat Algorithm

If $2^N \bmod N = 2$ then N has a high probability to be a prime number.

Example:

let $N=3$ (we know that 3 is a prime).

$2^3 \bmod 3 = 8 \bmod 3 = 2$, then N has a high probability to be a prime number... and in fact, it is really prime.

Another example:

let $N=11$ (we know that 11 is a prime).

$2^{11} \bmod 11 = 2048 \bmod 11 = 2$, then N has a high probability to be a prime number... again, this is also really prime.

Sieve of Eratosthenes:

Sieve is the best prime generator algorithm. It will generate a list of primes very quickly, but it will need a very big memory. You can use Boolean flags to do this (Boolean is only 1 byte).

Algorithm for Sieve of Eratosthenes to find the prime numbers within a range L,U (inclusive), where must be $L \leq U$.

```
void sieve(int L,int U) {
    int i,j,d;
    d=U-L+1; /* from range L to U, we have d=U-L+1 numbers. */
    /* use flag[i] to mark whether (L+i) is a prime number or not. */

    bool *flag=new bool[d];
    for (i=0;i<d;i++) flag[i]=true; /* default: mark all to be true */

    for (i=(L%2!=0);i<d;i+=2) flag[i]=false;

    /* sieve by prime factors staring from 3 till sqrt(U) */
    for (i=3;i<=sqrt(U);i+=2) {
        if (i>L && !flag[i-L]) continue;

        /* choose the first number to be sieved -- >=L,
           divisible by i, and not i itself! */
        j=L/i*i;    if (j<L) j+=i;
        if (j==i) j+=i; /* if j is a prime number, have to start form next
one */

        j-=L; /* change j to the index representing j */
        for (;j<d;j+=i) flag[j]=false;
    }

    if (L<=1) flag[1-L]=false;
    if (L<=2) flag[2-L]=true;

    for (i=0;i<d;i++) if (flag[i]) cout << (L+i) << " ";
    cout << endl;
}
```

CHAPTER 8 SORTING

Definition of a sorting problem

Input: A sequence of N numbers $(a_1, a_2, \dots, a_N)$

Output: A permutation $(a_{1'}, a_{2'}, \dots, a_{N'})$ of the input sequence such that $a_{1'} \leq a_{2'} \leq \dots \leq a_{N'}$

Things to be considered in Sorting

These are the difficulties in sorting that can also happen in real life:

- A. Size of the list to be ordered is the main concern. Sometimes, the computer memory is not sufficient to store all data. You may only be able to hold part of the data inside the computer at any time, the rest will probably have to stay on disc or tape. This is known as the problem of external sorting. However, rest assured, that almost all programming contests problem size will never be extremely big such that you need to access disc or tape to perform external sorting... (such hardware access is usually forbidden during contests).
- B. Another problem is the stability of the sorting method. Example: suppose you are an airline. You have a list of the passengers for the day's flights. Associated to each passenger is the number of his/her flight. You will probably want to sort the list into alphabetical order. No problem... Then, you want to re-sort the list by flight number so as to get lists of passengers for each flight. Again, "no problem"... - except that it would be very nice if, for each flight list, the names were still in alphabetical order. This is the problem of stable sorting.
- C. To be a bit more mathematical about it, suppose we have a list of items $\{x_i\}$ with x_a equal to x_b as far as the sorting comparison is concerned and with x_a before x_b in the list. The sorting method is stable if x_a is sure to come before x_b in the sorted list.

Finally, we have the problem of key sorting. The individual items to be sorted might be very large objects (e.g. complicated record cards). All sorting methods naturally involve a lot of moving around of the things being sorted. If the things are very large this might take up a lot of computing time -- much more than that taken just to switch two integers in an array.

Comparison-based sorting algorithms

Comparison-based sorting algorithms involves comparison between two object a and b to determine one of the three possible relationship between them: less than, equal, or greater

than. These sorting algorithms are dealing with how to use this comparison effectively, so that we minimize the amount of such comparison. Let's start from the most naive version to the most sophisticated comparison-based sorting algorithms.

Bubble Sort

Speed: $O(n^2)$, extremely slow

Space: The size of initial array

Coding Complexity: Simple

This is the simplest and (unfortunately) the worst sorting algorithm. This sort will do double pass on the array and swap 2 values when necessary.

```
BubbleSort(A)
  for i <- length[A]-1 down to 1
    for j <- 0 to i-1
      if (A[j] > A[j+1]) // change ">" to "<" to do a descending sort
        temp <- A[j]
        A[j] <- A[j+1]
        A[j+1] <- temp
```

Slow motion run of Bubble Sort (**Bold** == sorted region):

```
5 2 3 1 4
2 3 1 4 5
2 1 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5 >> done
```

TEST YOUR BUBBLE SORT KNOWLEDGE

Solve UVa problems related with Bubble sort:

[299 - Train Swapping](#)

[612 - DNA Sorting](#)

[10327 - Flip Sort](#)

Quick Sort

Speed: $O(n \log n)$, one of the best sorting algorithm.

Space: The size of initial array

Coding Complexity: Complex, using Divide & Conquer approach

One of the best sorting algorithm known. Quick sort use Divide & Conquer approach and partition each subset. Partitioning a set is to divide a set into a collection of mutually disjoint sets. This sort is much quicker compared to "stupid but simple" bubble sort. Quick sort was invented by C.A.R Hoare.

Quick Sort - basic idea

Partition the array in $O(n)$

Recursively sort left array in $O(\log_2 n)$ best/average case

Recursively sort right array in $O(\log_2 n)$ best/average case

Quick sort pseudo code:

```
QuickSort(A,p,r)
  if p < r
    q <- Partition(A,p,r)
    QuickSort(A,p,q)
    QuickSort(A,q+1,r)
```

Quick Sort for C/C++ User

C/C++ standard library <stdlib.h> contains qsort function.

This is not the best quick sort implementation in the world but it fast enough and VERY EASY to be used... therefore if you are using C/C++ and need to sort something, you can simply call this built in function:

```
qsort(<arrayname>,<size>,sizeof(<elementsize>),compare_function);
```

The only thing that you need to implement is the `compare_function`, which takes in two arguments of type "const void", which can be cast to appropriate data structure, and then return one of these three values:

- ⤴ negative, if a should be before b
- ⤴ 0, if a equal to b
- ⤴ positive, if a should be after b

1. Comparing a list of integers

simply cast a and b to integers

if $x < y$, $x - y$ is negative, $x == y$, $x - y = 0$, $x > y$, $x - y$ is positive

$x - y$ is a shortcut way to do it :)

reverse $*x - *y$ to $*y - *x$ for sorting in decreasing order

```
int compare_function(const void *a, const void *b) {
    int *x = (int *) a;
    int *y = (int *) b;
    return *x - *y;
}
```

2. Comparing a list of strings

For comparing string, you need `strcmp` function inside `string.h` lib. `strcmp` will by default return -ve, 0, ve appropriately... to sort in reverse order, just reverse the sign returned by `strcmp`

```
#include <string.h>

int compare_function(const void *a, const void *b) {
    return (strcmp((char *)a, (char *)b));
}
```

3. Comparing floating point numbers

```
int compare_function(const void *a, const void *b) {
    double *x = (double *) a;
    double *y = (double *) b;
    // return *x - *y; // this is WRONG...
    if (*x < *y) return -1;
    else if (*x > *y) return 1; return 0;
}
```

4. Comparing records based on a key

Sometimes you need to sort a more complex stuffs, such as record. Here is the simplest way to do it using `qsort` library

```
typedef struct {
    int key;
    double value;
} the_record;
```

```
int compare_function(const void *a,const void *b) {  
    the_record *x = (the_record *) a;  
    the_record *y = (the_record *) b;  
    return x->key - y->key;  
}
```

Multi field sorting, advanced sorting technique

Sometimes sorting is not based on one key only.

For example sorting birthday list. First you sort by month, then if the month ties, sort by date (obviously), then finally by year.

For example I have an unsorted birthday list like this:

```
24 - 05 - 1982 - Sunny  
24 - 05 - 1980 - Cecilia  
31 - 12 - 1999 - End of 20th century  
01 - 01 - 0001 - Start of modern calendar
```

I will have a sorted list like this:

```
01 - 01 - 0001 - Start of modern calendar  
24 - 05 - 1980 - Cecilia  
24 - 05 - 1982 - Sunny  
31 - 12 - 1999 - End of 20th century
```

To do multi field sorting like this, traditionally one will choose multiple sort using sorting algorithm which has "stable-sort" property.

The better way to do multi field sorting is to modify the `compare_function` in such a way that you break ties accordingly... I'll give you an example using birthday list again.

```
typedef struct {  
    int day,month,year;  
    char *name;  
} birthday;  
  
int compare_function(const void *a,const void *b) {  
    birthday *x = (birthday *) a;  
    birthday *y = (birthday *) b;  
  
    if (x->month != y->month) // months different  
        return x->month - y->month; // sort by month
```



```
else { // months equal..., try the 2nd field... day
    if (x->day != y->day) // days different

        return x->day - y->day; // sort by day
    else // days equal, try the 3rd field... year
        return x->year - y->year; // sort by year
}
```

TEST YOUR MULTI FIELD SORTING KNOWLEDGE

10194 - Football (aka Soccer)

Linear-time Sorting

a. Lower bound of comparison-based sort is $O(n \log n)$

The sorting algorithms that we see above are comparison-based sort, they use comparison function such as $<$, $\leq$, $=$, $>$, $\geq$, etc to compare 2 elements. We can model this comparison sort using decision tree model, and we can prove that the shortest height of this tree is $O(n \log n)$.

b. Counting Sort

For Counting Sort, we assume that the numbers are in the range $[0..k]$, where k is at most $O(n)$. We set up a counter array which counts how many duplicates inside the input, and the reorder the output accordingly, without any comparison at all. Complexity is $O(n+k)$.

c. Radix Sort

For Radix Sort, we assume that the input are n d -digits number, where d is reasonably limited.

Radix Sort will then sorts these number digit by digit, starting with the least significant digit to the most significant digit. It usually use a stable sort algorithm to sort the digits, such as Counting Sort above.

Example:

input:

321

257

113
622

sort by third (last) digit:

321
622
113
257

after this phase, the third (last) digit is sorted.

sort by second digit:

113
321
622
257

after this phase, the second and third (last) digit are sorted.

sort by first digit:

113
257
321
622

after this phase, all digits are sorted.

For a set of n d -digits numbers, we will do d pass of counting sort which have complexity $O(n+k)$, therefore, the complexity of Radix Sort is $O(d(n+k))$.

CHAPTER 9 SEARCHING

Searching is very important in computer science. It is very closely related to sorting. We usually search after sorting the data. Remember Binary Search? This is the best example of an algorithm which utilizes both Sorting and Searching.^[2]

Search algorithm depends on the data structure used. If we want to search a graph, then we have a set of well known algorithms such as DFS, BFS, etc

Binary Search

The most common application of binary search is to find a specific value in a sorted list. The search begins by examining the value in the center of the list; because the values are sorted, it then knows whether the value occurs before or after the center value, and searches through the correct half in the same way. Here is simple pseudocode which determines the index of a given value in a sorted list a between indices left and right:

```
function binarySearch(a, value, left, right)
    if right < left
        return not found
    mid := floor((left+right)/2)
    if a[mid] = value
        return mid
    if value < a[mid]
        binarySearch(a, value, left, mid-1)
    else
        binarySearch(a, value, mid+1, right)
```

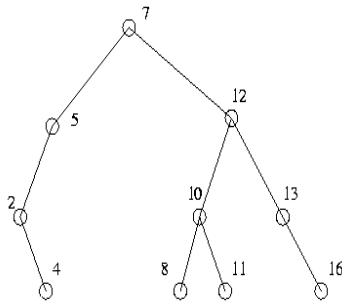
In both cases, the algorithm terminates because on each recursive call or iteration, the range of indexes right minus left always gets smaller, and so must eventually become negative.

Binary search is a logarithmic algorithm and executes in $O(\log n)$ time. Specifically, $1 + \log_2 N$ iterations are needed to return an answer. It is considerably faster than a linear search. It can be implemented using recursion or iteration, as shown above, although in many languages it is more elegantly expressed recursively.

Binary Search Tree

Binary Search Tree (BST) enable you to search a collection of objects (each with a real or integer value) quickly to determine if a given value exists in the collection.

Basically, a binary search tree is a node-weighted, rooted binary ordered tree. That collection of adjectives means that each node in the tree might have no child, one left child, one right child, or both left and right child. In addition, each node has an object associated with it, and the weight of the node is the value of the object.



The binary search tree also has the **property** that each node's left child and descendants of its left child have a value less than that of the node, and each node's right child and its descendants have a value greater or equal to it.

Binary Search Tree

The nodes are generally represented as a structure with four fields, a pointer to the node's left child, a pointer to the node's right child, the weight of the object stored at this node, and a pointer to the object itself. Sometimes, for easier access, people add pointer to the parent too.

Why are Binary Search Tree useful?

Given a collection of n objects, a binary search tree takes only $O(\text{height})$ time to find an objects, assuming that the tree is not really poor (unbalanced), $O(\text{height})$ is $O(\log n)$. In addition, unlike just keeping a sorted array, inserting and deleting objects only takes $O(\log n)$ time as well. You also can get all the keys in a Binary Search Tree in a sorted order by traversing it using $O(n)$ inorder traversal.

Variations on Binary Trees

There are several variants that ensure that the trees are never poor. Splay trees, Red-black trees, B-trees, and AVL trees are some of the more common examples. They are all much more complicated to code, and random trees are generally good, so it's generally not worth it.

Tips: If you're concerned that the tree you created might be bad (it's being created by inserting elements from an input file, for example), then randomly order the elements before insertion.

Dictionary

A dictionary, or hash table, stores data with a very quick way to do lookups. Let's say there is a collection of objects and a data structure must quickly answer the question: 'Is

this object in the data structure?' (e.g., is this word in the dictionary?). A hash table does this in less time than it takes to do binary search.

The idea is this: find a function that maps the elements of the collection to an integer between 1 and x (where x , in this explanation, is larger than the number of elements in your collection). Keep an array indexed from 1 to x , and store each element at the position that the function evaluates the element as. Then, to determine if something is in your collection, just plug it into the function and see whether or not that position is empty. If it is not check the element there to see if it is the same as the something you're holding,

For example, presume the function is defined over 3-character words, and is $(\text{first letter} + (\text{second letter} * 3) + (\text{third letter} * 7)) \bmod 11$ ($A=1$, $B=2$, etc.), and the words are 'CAT', 'CAR', and 'COB'. When using ASCII, this function takes 'CAT' and maps it to 3, maps 'CAR' to 0, and maps 'COB' to 7, so the hash table would look like this:

```
0: CAR
1
2
3: CAT
4
5
6
7: COB
8
9
10
```

Now, to see if 'BAT' is in there, plug it into the hash function to get 2. This position in the hash table is empty, so it is not in the collection. 'ACT', on the other hand, returns the value 7, so the program must check to see if that entry, 'COB', is the same as 'ACT' (no, so 'ACT' is not in the dictionary either). In the other hand, if the search input is 'CAR', 'CAT', 'COB', the dictionary will return true.

Collision Handling

This glossed over a slight problem that arises. What can be done if two entries map to the same value (e.g., we wanted to add 'ACT' and 'COB')? This is called a collision. There are couple ways to correct collisions, but this document will focus on one method, called chaining.

Instead of having one entry at each position, maintain a linked list of entries with the same hash value. Thus, whenever an element is added, find its position and add it to the

beginning (or tail) of that list. Thus, to have both 'ACT' and 'COB' in the table, it would look something like this:

```
0: CAR
1
2
3: CAT
4
5
6
7: COB -> ACT
8
9
10
```

Now, to check an entry, all elements in the linked list must be examined to find out the element is not in the collection. This, of course, decreases the efficiency of using the hash table, but it's often quite handy.

Hash Table variations

It is often quite useful to store more information than just the value. One example is when searching a small subset of a large subset, and using the hash table to store locations visited, you may want the value for searching a location in the hash table with it.

CHAPTER 10 GREEDY ALGORITHMS

Greedy algorithms are algorithms which follow the problem solving meta-heuristic of making the locally optimum choice at each stage with the hope of finding the global optimum. For instance, applying the greedy strategy to the traveling salesman problem yields the following algorithm: "At each stage visit the nearest unvisited city to the current city".[Wiki Encyclopedia]

Greedy algorithms do not consistently find the globally optimal solution, because they usually do not operate exhaustively on all the data. They can make commitments to certain choices too early which prevent them from finding the best overall solution later. For example, all known greedy algorithms for the graph coloring problem and all other NP-complete problems do not consistently find optimum solutions. Nevertheless, they are useful because they are quick to think up and often give good approximations to the optimum.

If a greedy algorithm can be proven to yield the global optimum for a given problem class, it typically becomes the method of choice. Examples of such greedy algorithms are Kruskal's algorithm and Prim's algorithm for finding minimum spanning trees and the algorithm for finding optimum Huffman trees. The theory of matroids, as well as the even more general theory of greedoids, provide whole classes of such algorithms.

In general, greedy algorithms have five pillars:

- ◆ A candidate set, from which a solution is created
- ◆ A selection function, which chooses the best candidate to be added to the solution
- ◆ A feasibility function, that is used to determine if a candidate can be used to contribute to a solution
- ◆ An objective function, which assigns a value to a solution, or a partial solution, and
- ◆ A solution function, which will indicate when we have discovered a complete solution

Barn Repair

There is a long list of stalls, some of which need to be covered with boards. You can use up to N ($1 \leq N \leq 50$) boards, each of which may cover any number of consecutive stalls. Cover all the necessary stalls, while covering as few total stalls as possible. How will you solve it?

The basic idea behind greedy algorithms is to build large solutions up from smaller ones. Unlike other approaches, however, greedy algorithms keep only the best solution they find as they go along. Thus, for the sample problem, to build the answer for $N = 5$, they find the best solution for $N = 4$, and then alter it to get a solution for $N = 5$. No other solution for $N = 4$ is ever considered.

Greedy algorithms are fast, generally linear to quadratic and require little extra memory. Unfortunately, they usually aren't correct. But when they do work, they are often easy to implement and fast enough to execute.

Problems with Greedy algorithms

There are two basic problems to greedy algorithms.

1. How to Build

How does one create larger solutions from smaller ones? In general, this is a function of the problem. For the sample problem, the most obvious way to go from four boards to five boards is to pick a board and remove a section, thus creating two boards from one. You should choose to remove the largest section from any board which covers only stalls which don't need covering (so as to minimize the total number of stalls covered).

To remove a section of covered stalls, take the board which spans those stalls, and make into two boards: one of which covers the stalls before the section, one of which covers the stalls after the second.

2. Does it work?

The real challenge for the programmer lies in the fact that greedy solutions don't always work. Even if they seem to work for the sample input, random input, and all the cases you can think of, if there's a case where it won't work, at least one (if not more!) of the judges' test cases will be of that form.

For the sample problem, to see that the greedy algorithm described above works, consider the following:

Assume that the answer doesn't contain the large gap which the algorithm removed, but does contain a gap which is smaller. By combining the two boards at the end of the smaller gap and splitting the board across the larger gap, an answer is obtained which uses as many boards as the original solution but which covers fewer stalls. This new answer is better, so therefore the assumption is wrong and we should always choose to remove the largest gap.

If the answer doesn't contain this particular gap but does contain another gap which is just as large, doing the same transformation yields an answer which uses as many boards and covers as many stalls as the other answer. This new answer is just as good as the original solution but no better, so we may choose either.

Thus, there exists an optimal answer which contains the large gap, so at each step, there is always an optimal answer which is a superset of the current state. Thus, the final answer is optimal.

If a greedy solution exists, use it. They are easy to code, easy to debug, run quickly, and use little memory, basically defining a good algorithm in contest terms. The only missing element from that list is correctness. If the greedy algorithm finds the correct answer, go for it, but don't get suckered into thinking the greedy solution will work for all problems.

Sorting a three-valued sequence

You are given a three-valued (1, 2, or 3) sequence of length up to 1000. Find a minimum set of exchanges to put the sequence in sorted order.

The sequence has three parts: the part which will be 1 when in sorted order, 2 when in sorted order, and 3 when in sorted order. The greedy algorithm swaps as many as possible of the 1's in the 2 part with 2's in the 1 part, as many as possible 1's in the 3 part with 3's in the 1 part, and 2's in the 3 part with 3's in the 2 part. Once none of these types remains, the remaining elements out of place need to be rotated one way or the other in sets of 3. You can optimally sort these by swapping all the 1's into place and then all the 2's into place.

Analysis: Obviously, a swap can put at most two elements in place, so all the swaps of the first type are optimal. Also, it is clear that they use different types of elements, so there is no "interference" between those types. This means the order does not matter. Once those swaps have been performed, the best you can do is two swaps for every three elements not in the correct location, which is what the second part will achieve (for example, all the 1's are put in place but no others; then all that remains are 2's in the 3's place and vice-versa, and which can be swapped).

Topological Sort

Given a collection of objects, along with some ordering constraints, such as "A must be before B," find an order of the objects such that all the ordering constraints hold.

Algorithm: Create a directed graph over the objects, where there is an arc from A to B if "A must be before B." Make a pass through the objects in arbitrary order. Each time you find an object with in-degree of 0, greedily place it on the end of the current ordering, delete all of its out-arcs, and recurse on its (former) children, performing the same check. If this algorithm gets through all the objects without putting every object in the ordering, there is no ordering which satisfies the constraints.

TEST YOUR GREEDY KNOWLEDGE

Solve UVa problems related which utilizes Greedy algorithms:

[10020 - Minimal Coverage](#)

[10340 - All in All](#)

[10440 - Ferry Loading \(II\)](#)

CHAPTER 11 DYNAMIC PROGRAMMING

Dynamic Programming (shortened as DP) is a programming technique that can dramatically reduce the runtime of some algorithms (but not all problem has DP characteristics) from exponential to polynomial. Many (and still increasing) real world problems are only solvable within reasonable time using DP.

To be able to use DP, the original problem must have:

1. **Optimal sub-structure** property:
Optimal solution to the problem contains within it optimal solutions to sub-problems
2. **Overlapping sub-problems** property
We accidentally recalculate the same problem twice or more.

There are 2 types of DP: We can either build up solutions of sub-problems from small to large (bottom up) or we can save results of solutions of sub-problems in a table (top down + memoization).

Let's start with a sample of Dynamic Programming (DP) technique. We will examine the simplest form of overlapping sub-problems. Remember Fibonacci? A popular problem which creates a lot of redundancy if you use standard recursion $f_n = f_{n-1} + f_{n-2}$.

Top-down Fibonacci DP solution will record each Fibonacci calculation in a table so it won't have to re-compute the value again when you need it, a simple table-lookup is enough (memorization), whereas Bottom-up DP solution will build the solution from smaller numbers.

Now let's see the comparison between Non-DP solution versus DP solution (both bottom-up and top-down), given in the C source code below, along with the appropriate comments

```
#include <stdio.h>

#define MAX 20 // to test with bigger number, adjust this value

int memo[MAX]; // array to store the previous calculations

// the slowest, unnecessary computation is repeated
int Non_DP(int n) {
    if (n==1 || n==2)
        return 1;
```

```
else
    return Non_DP(n-1) + Non_DP(n-2);
}

// top down DP
int DP_Top_Down(int n) {
    // base case
    if (n == 1 || n == 2)
        return 1;

    // immediately return the previously computed result
    if (memo[n] != 0)
        return memo[n];

    // otherwise, do the same as Non_DP
    memo[n] = DP_Top_Down(n-1) + DP_Top_Down(n-2);
    return memo[n];
}

// fastest DP, bottom up, store the previous results in array
int DP_Bottom_Up(int n) {
    memo[1] = memo[2] = 1; // default values for DP algorithm

    // from 3 to n (we already know that fib(1) and fib(2) = 1
    for (int i=3; i<=n; i++)
        memo[i] = memo[i-1] + memo[i-2];

    return memo[n];
}

void main() {
    int z;

    // this will be the slowest
    for (z=1; z<MAX; z++) printf("%d-", Non_DP(z));
    printf("\n\n");

    // this will be much faster than the first
    for (z=0; z<MAX; z++) memo[z] = 0;
    for (z=1; z<MAX; z++) printf("%d-", DP_Top_Down(z));
    printf("\n\n");

    /* this normally will be the fastest */
    for (z=0; z<MAX; z++) memo[z] = 0;
    for (z=1; z<MAX; z++) printf("%d-", DP_Bottom_Up(z));
    printf("\n\n");
}
```

Matrix Chain Multiplication (MCM)

Let's start by analyzing the cost of multiplying 2 matrices:

```

Matrix-Multiply(A,B):
  if columns[A] != columns[B] then
    error "incompatible dimensions"
  else
    for i = 1 to rows[A] do
      for j = 1 to columns[B] do
        C[i,j]=0
        for k = 1 to columns[A] do
          C[i,j] = C[i,j] + A[i,k] * B[k,j]
        return C

```

Time complexity = $O(pqr)$ where $|A|=p \times q$ and $|B|=q \times r$

$|A| = 2 \times 3$, $|B| = 3 \times 1$, therefore to multiply these 2 matrices, we need $O(2 \times 3 \times 1) = O(6)$

scalar multiplication. The result is matrix C with $|C| = 2 \times 1$

TEST YOUR MATRIX MULTIPLICATION KNOWLEDGE

Solve UVa problems related with Matrix Multiplication:

[442 - Matrix Chain Multiplication](#) - Straightforward problem

Matrix Chain Multiplication Problem

Input: Matrices $A_1, A_2, \dots, A_n$, each A_i of size $P_{i-1} \times P_i$

Output: Fully parenthesized product $A_1 A_2 \dots A_n$ that minimizes the number of scalar multiplications

A product of matrices is fully parenthesized if it is either

1. a single matrix
2. the product of 2 fully parenthesized matrix products surrounded by parentheses

Example of MCM problem:

We have 3 matrices and the size of each matrix:

A_1 (10 x 100), A_2 (100 x 5), A_3 (5 x 50)

We can fully parenthesize them in two ways:

1. $(A_1 (A_2 A_3)) = 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$
2. $((A_1 A_2) A_3) = 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ (10 times better)

See how the cost of multiplying these 3 matrices differ significantly. The cost truly depend on the choice of the fully parenthesization of the matrices. However, exhaustively checking all possible parenthesizations take exponential time.

Now let's see how MCM problem can be solved using DP.

Step 1: characterize the optimal sub-structure of this problem.

Let $A_{i..j}$ ($i < j$) denote the result of multiplying $A_i A_{i+1} \dots A_j$. $A_{i..j}$ can be obtained by splitting it into $A_{i..k}$ and $A_{k+1..j}$ and then multiplying the sub-products. There are $j-i$ possible splits (i.e. $k=i, \dots, j-1$)

Within the optimal parenthesization of $A_{i..j}$:

- (a) the parenthesization of $A_{i..k}$ must be optimal
- (b) the parenthesization of $A_{k+1..j}$ must be optimal

Because if they are not optimal, then there exist other split which is better, and we should choose that split and not this split.

Step 2: Recursive formulation

Need to find $A_{1..n}$

Let $m[i..j]$ = minimum number of scalar multiplications needed to compute $A_{i..j}$

Since $A_{i..j}$ can be obtained by breaking it into $A_{i..k}$ $A_{k+1..j}$, we have

$$m[i..j] = 0, \text{ if } i=j$$

$$= \min_{i \leq k < j} \{ m[i..k] + m[k+1..j] + p_{i-1} p_k p_j \}, \text{ if } i < j$$

let $s[i..j]$ be the value k where the optimal split occurs.

Step 3 Computing the Optimal Costs

```
Matric-Chain-Order(p)
n = length[p]-1
```

```

for i = 1 to n do
  m[i,i] = 0
for l = 2 to n do
  for i = 1 to n-l+1 do
    j = i+l-1
    m[i,j] = infinity
    for k = i to j-1 do
      q = m[i,k] + m[k+1,j] + pi-1*pk*pj
      if q < m[i,j] then
        m[i,j] = q
        s[i,j] = k
return m and s

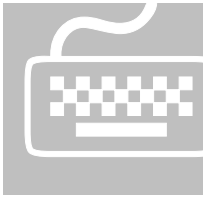
```

Step 4: Constructing an Optimal Solution

```

Print-MCM(s,i,j)
  if i=j then
    print Ai
  else
    print "(" + Print-MCM(s,l,s[i,j]) + "*" + Print-MCM(s,s[i,j]+1,j) +
    ")"

```



Note: As any other DP solution, MCM also can be solved using Top Down recursive algorithm using memoization. Sometimes, if you cannot visualize the Bottom Up, approach, just modify your original Top Down recursive solution by including memoization. You'll save a lot of time by avoiding repetitive calculation of sub-problems.

TEST YOUR MATRIX CHAIN MULTIPLICATION KNOWLEDGE

Solve UVA problems related with Matrix Chain Multiplication:

348 - Optimal Array Multiplication Sequence - Use algorithm above

Longest Common Subsequence (LCS)

Input: Two sequence

Output: A longest common subsequence of those two sequences, see details below.

A sequence Z is a **subsequence** of $X \langle x_1, x_2, \dots, x_m \rangle$, if there exists a strictly increasing sequence $\langle i_1, i_2, \dots, i_k \rangle$ of indices of X such that for all $j=1, 2, \dots, k$, we have $x_{i_j} = z_j$. example: $X = \langle B, C, A, D \rangle$ and $Z = \langle C, A \rangle$.

A sequence Z is called **common subsequence** of sequence X and Y if Z is subsequence of both X and Y.

longest common subsequence (LCS) is just the longest "common subsequence" of two sequences.

A brute force approach of finding LCS such as enumerating all subsequences and finding the longest common one takes too much time. However, Computer Scientist has found a Dynamic Programming solution for LCS problem, we will only write the final code here, written in C, ready to use. Note that this code is slightly modified and we use global variables (yes this is not Object Oriented).

```
#include <stdio.h>
#include <string.h>
#define MAX 100

char X[MAX], Y[MAX];
int i, j, m, n, c[MAX][MAX], b[MAX][MAX];

int LCSlength() {

    m=strlen(X);
    n=strlen(Y);

    for (i=1; i<=m; i++) c[i][0]=0;
    for (j=0; j<=n; j++) c[0][j]=0;

    for (i=1; i<=m; i++)
        for (j=1; j<=n; j++) {
            if (X[i-1]==Y[j-1]) {
                c[i][j]=c[i-1][j-1]+1;
                b[i][j]=1; /* from north west */
            }

            else if (c[i-1][j]>=c[i][j-1]) {
                c[i][j]=c[i-1][j];

                b[i][j]=2; /* from north */
            }

            else {
                c[i][j]=c[i][j-1];
                b[i][j]=3; /* from west */
            }
        }
    return c[m][n];
}
```



```
void printLCS(int i,int j) {
    if (i==0 || j==0) return;

    if (b[i][j]==1) {
        printLCS(i-1,j-1);
        printf("%c",X[i-1]);}
    else if (b[i][j]==2)
        printLCS(i-1,j);
    else
        printLCS(i,j-1);
}

void main() {
    while (1) {
        gets(X);
        if (feof(stdin)) break; /* press ctrl+z to terminate */
        gets(Y);
        printf("LCS length -> %d\n",LCSlength()); /* count length */
        printLCS(m,n); /* reconstruct LCS */
        printf("\n");
    }
}
```

TEST YOUR LONGEST COMMON SUBSEQUENCE KNOWLEDGE

Solve UVa problems related with LCS:

[531 - Compromise](#)

[10066 - The Twin Towers](#)

[10100 - Longest Match](#)

[10192 - Vacation](#)

[10405 - Longest Common Subsequence](#)

Edit Distance

Input: Given two string, Cost for deletion, insertion, and replace

Output: Give the minimum actions needed to transform first string into the second one.

Edit Distance problem is a bit similar to LCS. DP Solution for this problem is very useful in Computational Biology such as for comparing DNA.

Let $d(\text{string1}, \text{string2})$ be the distance between these 2 strings.

A two-dimensional matrix, $m[0..|s1|, 0..|s2|]$ is used to hold the edit distance values, such that $m[i,j] = d(s1[1..i], s2[1..j])$.

```

m[0][0] = 0;
for (i=1; i<length(s1); i++) m[i][0] = i;
for (j=1; j<length(s2); j++) m[0][j] = j;

for (i=0; i<length(s1); i++)
  for (j=0; j<length(s2); j++) {
    val = (s1[i] == s2[j]) ? 0 : 1;
    m[i][j] = min( m[i-1][j-1] + val,
                  min(m[i-1][j]+1 , m[i][j-1]+1));
  }

```

To output the trace, use another array to store our action along the way. Trace back these values later.

TEST YOUR EDIT DISTANCE KNOWLEDGE

[164 - String Computer](#)

[526 - String Distance and Edit Process](#)

Longest Inc/Dec-reasing Subsequence (LIS/LDS)

Input: Given a sequence

Output: The longest subsequence of the given sequence such that all values in this longest subsequence is strictly increasing/decreasing.

$O(N^2)$ DP solution for LIS problem (this code check for increasing values):

```

for i = 1 to total-1
  for j = i+1 to total
    if height[j] > height[i] then
      if length[i] + 1 > length[j] then
        length[j] = length[i] + 1
        predecessor[j] = i

```

Example of LIS

height sequence: 1,6,2,3,5

length initially: [1,1,1,1,1] - because max length is at least 1 rite...

predecessor initially: [nil,nil,nil,nil,nil] - assume no predecessor so far

```

After first loop of j:
  length: [1,2,2,2,2], because 6,2,3,5 are all > 1
  predecessor: [nil,1,1,1,1]
After second loop of j: (No change)
  length: [1,2,2,2,2], because 2,3,5 are all < 6
  predecessor: [nil,1,1,1,1]
After third loop:
  length: [1,2,2,3,3], because 3,5 are all > 2
  predecessor: [nil,1,1,3,3]

  After fourth loop:
  length: [1,2,2,3,4], because 5 > 3
  predecessor: [nil,1,1,3,4]

```

We can reconstruct the solution using recursion and predecessor array.

Is $O(n^2)$ is the best algorithm to solve LIS/LDS ?

Fortunately, the answer is “No”.

There exist an $O(n \log k)$ algorithm to compute LIS (for LDS, this is just a reversed-LIS), where k is the size of the actual LIS.

This algorithm use some invariant, where for each longest subsequence with length l , it will terminate with value $A[l]$. (Notice that by maintaining this invariant, array A will be naturally sorted.) Subsequent insertion (you will only do n insertions, one number at one time) will use binary search to find the appropriate position in this sorted array A

```

0  1  2  3  4  5  6  7  8
a  -7,10, 9, 2, 3, 8, 8, 1

A -i  i, i, i, i, i, i, i, i, i (iteration number, i = infinity)
A -i -7, i, i, i, i, i, i, i, i (1)
A -i -7,10, i, i, i, i, i, i, i (2)
A -i -7, 9, i, i, i, i, i, i, i (3)
A -i -7, 2, i, i, i, i, i, i, i (4)
A -i -7, 2, 3, i, i, i, i, i, i (5)
A -i -7, 2, 3, 8, i, i, i, i, i (6)
A -i -7, 2, 3, 8, i, i, i, i, i (7)
A -i -7, 1, 3, 8, i, i, i, i, i (8)

```

You can see that the length of LIS is 4, which is correct. To reconstruct the LIS, at each step, store the predecessor array as in standard LIS + this time remember the actual values, since array A only store the last element in the subsequence, not the actual values.

TEST YOUR LONGEST INC/DEC-REASING SUBSEQUENCE KNOWLEDGE

[111 - History Grading](#)
[231 - Testing the CATCHER](#)
[481 - What Goes Up](#) - need $O(n \log k)$ LIS
[497 - Strategic Defense Initiative](#)
[10051 - Tower of Cubes](#)
[10131 - Is Bigger Smarter](#)

Zero-One Knapsack

Input: N items, each with various V_i (Value) and W_i (Weight) and max Knapsack size MW.

Output: Maximum value of items that one can carry, if he can either take or not-take a particular item.

Let $C[i][w]$ be the maximum value if the available items are $\{X_1, X_2, \dots, X_i\}$ and the knapsack size is w .

- ▲ if $i == 0$ or $w == 0$ (if no item or knapsack full), we can't take anything $C[i][w] = 0$
- ▲ if $W_i > w$ (this item too heavy for our knapsack), skip this item $C[i][w] = C[i-1][w]$;
- ▲ if $W_i \leq w$, take the maximum of "not-take" or "take" $C[i][w] = \max(C[i-1][w], C[i-1][w-W_i]+V_i)$;
- ▲ The solution can be found in $C[N][W]$;

```

for (i=0; i<=N; i++) C[i][0] = 0;
for (w=0; w<=MW; w++) C[0][w] = 0;

for (i=1; i<=N; i++)
    for (w=1; w<=MW; w++) {
        if (W[i] > w)
            C[i][w] = C[i-1][w];
        else
            C[i][w] = max(C[i-1][w], C[i-1][w-W[i]]+V[i]);
    }
output(C[N][MW]);

```

TEST YOUR 0-1 KNAPSACK KNOWLEDGE

Solve UVa problems related with 0-1 Knapsack:

[10130 - SuperSale](#)

Counting Change

Input: A list of denominations and a value N to be changed with these denominations

Output: Number of ways to change N

Suppose you have coins of 1 cent, 5 cents and 10 cents. You are asked to pay 16 cents, therefore you have to give 1 one cent, 1 five cents, and 1 ten cents. Counting Change algorithm can be used to determine how many ways you can use to pay an amount of money.

The number of ways to change amount A using N kinds of coins equals to:

1. The number of ways to change amount A using all but the first kind of coins, +
2. The number of ways to change amount A-D using all N kinds of coins, where D is the denomination of the first kind of coin.

The tree recursive process will gradually reduce the value of A, then using this rule, we can determine how many ways to change coins.

1. If A is exactly 0, we should count that as 1 way to make change.
2. If A is less than 0, we should count that as 0 ways to make change.
3. If N kinds of coins is 0, we should count that as 0 ways to make change.

```
#include <stdio.h>
#define MAXTOTAL 10000
long long nway[MAXTOTAL+1];

int coin[5] = { 50,25,10,5,1 };

void main()
{
    int i,j,n,v,c;
    scanf("%d",&n);
    v = 5;
    nway[0] = 1;
    for (i=0; i<v; i++) {
        c = coin[i];
        for (j=c; j<=n; j++)
            nway[j] += nway[j-c];
    }
    printf("%lld\n",nway[n]);
}
```

TEST YOUR COUNTING CHANGE KNOWLEDGE

[147 - Dollars](#)

[357 - Let Me Count The Ways](#) - Must use Big Integer [674 - Coin Change](#)

Maximum Interval Sum

Input: A sequence of integers

Output: A sum of an interval starting from index i to index j (consecutive), this sum must be maximum among all possible sums.

Numbers : -1 6

Sum : -1 6

^

max sum

Numbers : 4 -5 4 -3 4 4 -4 4 -5

Sum : 4 -1 4 1 5 9 5 9 4

^

^

stop

max sum

Numbers : -2 -3 -4

Sum : -2 -3 -4

^

max sum, but negative... (this is the maximum anyway)

So, just do a linear sweep from left to right, accumulate the sum one element by one element, start new interval whenever you encounter partial sum < 0 (and record current best maximum interval encountered so far)...

At the end, output the value of the maximum intervals.

TEST YOUR MAXIMUM INTERVAL SUM KNOWLEDGE

Solve UVa problems related with Maximum Interval Sum:

[507 - Jill Rides Again](#)

Other Dynamic Programming Algorithms

Problems that can be solved using Floyd Warshall and its variant, which belong to the category all-pairs shortest path algorithm, can be categorized as Dynamic Programming solution. Explanation regarding Floyd Warshall can be found in Graph section.

Other than that, there are a lot of ad hoc problems that can utilize DP, just remember that when the problem that you encountered exploits optimal sub-structure and repeating sub-problems, apply DP techniques, it may be helpful.

TEST YOUR DP KNOWLEDGE

Solve UVA problems related with Ad-Hoc DP:

[108 - Maximum Sum](#)

[836 - Largest Submatrix](#)

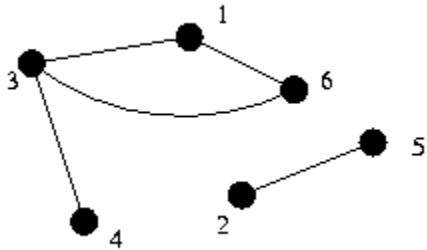
[10003 - Cutting Sticks](#)

[10465 - Homer Simpson](#)

CHAPTER 12 GRAPHS

A *graph* is a collection of *vertices* V and a collection of *edges* E consisting of pairs of vertices. Think of vertices as "locations". The set of vertices is the set of all the possible locations. In this analogy, edges represent paths between pairs of those locations. The set E contains all the paths between the locations.^[2]

Vertices and Edges



The graph is normally represented using that analogy. Vertices are points or circles, edges are lines between them.

In this example graph:

$V = \{1, 2, 3, 4, 5, 6\}$

$E = \{(1,3), (1,6), (2,5), (3,4), (3,6)\}$.

Each *vertex* is a member of the set V . A vertex is sometimes called a *node*.

Each *edge* is a member of the set E . Note that some vertices might not be the end point of any edge. Such vertices are termed "*isolated*".

Sometimes, numerical values are associated with edges, specifying lengths or costs; such graphs are called *edge-weighted* graphs (or *weighted* graphs). The value associated with an edge is called the *weight* of the edge. A similar definition holds for node-weighted graphs.

Telecommunication

Given a set of computers and a set of wires running between pairs of computers, what is the minimum number of machines whose crash causes two given machines to be unable to communicate? (The two given machines will not crash.)

Graph: The vertices of the graph are the computers. The edges are the wires between the computers. Graph problem: minimum dominating sub-graph.

Riding The Fences

Farmer John owns a large number of fences, which he must periodically check for integrity. He keeps track of his fences by maintaining a list of points at which fences intersect. He records the name of the point and the one or two fence names that touch that point. Every fence has two end points, each at some intersection point, although the intersection point may be the end point of only one fence.

Given a fence layout, calculate if there is a way for Farmer John to ride his horse to all of his fences without riding along a fence more than once. Farmer John can start and finish anywhere, but cannot cut across his fields (i.e., the only way he can travel between intersection points is along a fence). If there is a way, find one way.

Graph: Farmer John starts at intersection points and travels between the points along fences. Thus, the vertices of the underlying graph are the intersection points, and the fences represent edges. Graph problem: Traveling Salesman Problem.

Knight moves

Two squares on an 8x8 chessboard. Determine the shortest sequence of knight moves from one square to the other.

Graph: The graph here is harder to see. Each location on the chessboard represents a vertex. There is an edge between two positions if it is a legal knight move. Graph Problem: Single Source Shortest Path.

Overfencing

Farmer John created a huge maze of fences in a field. He omitted two fence segments on the edges, thus creating two "exits" for the maze. The maze is a "perfect" maze; you can find a way out of the maze from any point inside it.

Given the layout of the maze, calculate the number of steps required to exit the maze from the "worst" point in the maze (the point that is "farther" from either exit when walking optimally to the closest exit).

Here's what one particular $W=5$, $H=3$ maze looks like:

```

+-+--+--+
|      |
+-+ +-+ +-+
|  |||
+ +-+ +-+
||  |
+-+ +-+ +-+

```

Graph: The vertices of the graph are positions in the grid. There is an edge between two vertices if they represent adjacent positions that are not separated by a wall.

Terminology

An edge is a self-loop if it is of the form (u,u) . The sample graph contains no self-loops.

A graph is simple if it neither contains self-loops nor contains an edge that is repeated in E . A graph is called a multigraph if it contains a given edge more than once or contains self-loops. For our discussions, graphs are assumed to be simple. The example graph is a simple graph.

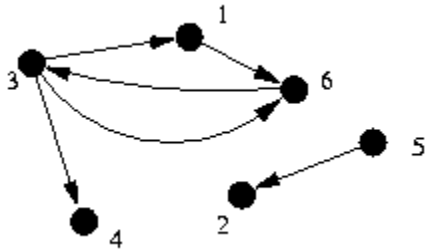
An edge (u,v) is incident to both vertex u and vertex v . For example, the edge $(1,3)$ is incident to vertex 3.

The degree of a vertex is the number of edges which are incident to it. For example, vertex 3 has degree 3, while vertex 4 has degree 1.

Vertex u is adjacent to vertex v if there is some edge to which both are incident (that is, there is an edge between them). For example, vertex 2 is adjacent to vertex 5.

A graph is said to be sparse if the total number of edges is small compared to the total number possible $((N \times (N-1))/2)$ and dense otherwise. For a given graph, whether it is dense or sparse is not well-defined.

Directed Graph



Graphs described thus far are called undirected, as the edges go 'both ways'. So far, the graphs have connoted that if one can travel from vertex 1 to vertex 3, one can also travel from vertex 1 to vertex 3. In other words, $(1,3)$ being in the edge set implies $(3,1)$ is in the edge set.

Sometimes, however, a graph is directed, in which case the edges have a direction. In this

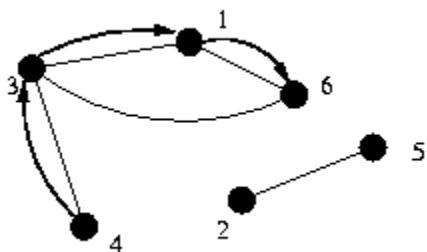
case, the edges are called arcs.

Directed graphs are drawn with arrows to show direction.

The out-degree of a vertex is the number of arcs which begin at that vertex. The in-degree of a vertex is the number of arcs which end at that vertex. For example, vertex 6 has in-degree 2 and out-degree 1.

A graph is assumed to be undirected unless specifically called a directed graph.

Paths



A path from vertex u to vertex x is a sequence of vertices $(v_0, v_1, \dots, v_k)$ such that $v_0 = u$ and $v_k = x$ and (v_0, v_1) is an edge in the graph, as is (v_1, v_2) , (v_2, v_3) , etc. The length of such a path is k .

For example, in the undirected graph above, $(4, 3, 1, 6)$ is a path.

This path is said to contain the vertices v_0, v_1 , etc., as well as the edges (v_0, v_1) , (v_1, v_2) , etc.

Vertex x is said to be *reachable* from vertex u if a path exists from u to x .

A path is *simple* if it contains no vertex more than once.

A path is a *cycle* if it is a path from some vertex to that same vertex. A cycle is *simple* if it contains no vertex more than once, except the start (and end) vertex, which only appears as the first and last vertex in the path.

These definitions extend similarly to directed graphs (e.g., (v_0, v_1) , (v_1, v_2) , etc. must be arcs).

Graph Representation

The choice of representation of a graph is important, as different representations have very different time and space costs.

The vertices are generally tracked by numbering them, so that one can index them just by their number. Thus, the representations focus on how to store the edges.

Edge List

The most obvious way to keep track of the edges is to keep a list of the pairs of vertices representing the edges in the graph.

This representation is easy to code, fairly easy to debug, and fairly space efficient. However, determining the edges incident to a given vertex is expensive, as is determining if two vertices are adjacent. Adding an edge is quick, but deleting one is difficult if its location in the list is not known.

For weighted graphs, this representation also keeps one more number for each edge, the edge weight. Extending this data structure to handle directed graphs is straightforward. Representing multigraphs is also trivial.

Example

	V1	V2
e1	4	3
e2	1	3
e3	2	5
e4	6	1
e5	3	6

Adjacency Matrix

A second way to represent a graph utilized an *adjacency matrix*. This is a N by N array (N is the number of vertices). The i,j entry contains a 1 if the edge (i,j) is in the graph; otherwise it contains a 0. For an undirected graph, this matrix is symmetric.

This representation is easy to code. It's much less space efficient, especially for large, sparse graphs. Debugging is harder, as the matrix is large. Finding all the edges incident to a given vertex is fairly expensive (linear in the number of vertices), but checking if two vertices are adjacent is very quick. Adding and removing edges are also very inexpensive operations.

For weighted graphs, the value of the (i,j) entry is used to store the weight of the edge. For an unweighted multigraph, the (i,j) entry can maintain the number of edges between the vertices. For a weighted multigraph, it's harder to extend this.

Example

The sample undirected graph would be represented by the following adjacency matrix:

	V1	V2	V3	V4	V5	V6
V1	0	0	1	0	0	1
V2	0	0	0	0	1	0
V3	1	0	0	1	0	1
V4	0	0	1	0	0	0
V5	0	1	0	0	0	0
V6	1	0	1	0	0	0

It is sometimes helpful to use the fact that the (i,j) entry of the adjacency matrix raised to the k -th power gives the number of paths from vertex i to vertex j consisting of exactly k edges.

Adjacency List

The third representation of a matrix is to keep track of all the edges incident to a given vertex. This can be done by using an array of length N , where N is the number of vertices. The i -th entry in this array is a list of the edges incident to i -th vertex (edges are represented by the index of the other vertex incident to that edge).

This representation is much more difficult to code, especially if the number of edges incident to each vertex is not bounded, so the lists must be linked lists (or dynamically allocated). Debugging this is difficult, as following linked lists is more difficult. However, this representation uses about as much memory as the edge list. Finding the vertices adjacent to each node is very cheap in this structure, but checking if two vertices are adjacent requires checking all the edges adjacent to one of the vertices. Adding an edge is easy, but deleting an edge is difficult, if the locations of the edge in the appropriate lists are not known.

Extend this representation to handle weighted graphs by maintaining both the weight and the other incident vertex for each edge instead of just the other incident vertex. Multigraphs are already representable. Directed graphs are also easily handled by this representation, in one of several ways: store only the edges in one direction, keep a separate list of incoming and outgoing arcs, or denote the direction of each arc in the list.

Example

The adjacency list representation of the example undirected graph is as follows:

Vertex	Adjacent Vertices
1	3, 6
2	5
3	6, 4, 1
4	3
5	2
6	3, 1

Implicit Representation

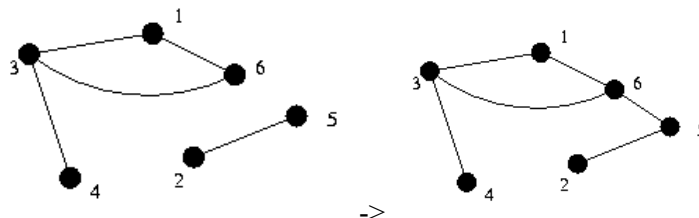
For some graphs, the graph itself does not have to be stored at all. For example, for the Knight moves and Overfencing problems, it is easy to calculate the neighbors of a vertex, check adjacency, and determine all the edges without actually storing that information, thus, there is no reason to actually store that information; the graph is implicit in the data itself.

If it is possible to store the graph in this format, it is generally the correct thing to do, as it saves a lot on storage and reduces the complexity of your code, making it easy to both write and debug.

If N is the number of vertices, M the number of edges, and $d_{\max}$ the maximum degree of a node, the following table summarizes the differences between the representations:

Efficiency	Edge List	Adj Matrix	Adj List
Space	$2 \cdot M$	N^2	$2 \cdot M$
Adjacency Check	M	1	$d_{\max}$
List of Adjacent Vertices	M	N	$d_{\max}$
Add Edge	1	1	1
Delete Edge	M	2	$2 \cdot d_{\max}$

Connectedness



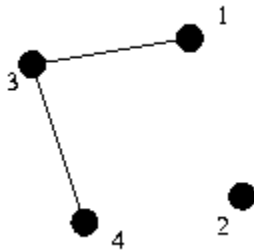
An undirected graph is said to be connected if there is a path from every vertex to every other vertex. The example graph is not connected, as there is no path from vertex 2 to vertex 4. However, if you add an edge between vertex 5 and vertex 6, then the graph becomes connected.

A component of a graph is a maximal subset of the vertices such that every vertex is reachable from each other vertex in the component. The original example graph has two components: $\{1, 3, 4, 6\}$ and $\{2, 5\}$. Note that $\{1, 3, 4\}$ is not a component, as it is not maximal.

A directed graph is said to be strongly connected if there is a path from every vertex to every other vertex.

A strongly connected component of a directed graph is a vertex u and the collection of all vertices v such that there is a path from u to v and a path from v to u .

Subgraphs

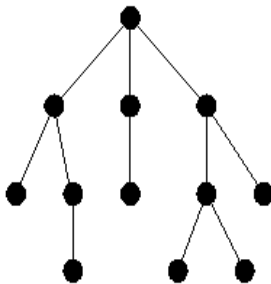


Graph $G' = (V', E')$ is a subgraph of $G = (V, E)$ if V' is a subset of V and E' is a subset of E .

The subgraph of G *induced* by V' is the graph (V', E') , where E' consists of all the edges of E that are between members of V' .

For example, for $V' = \{1, 3, 4, 2\}$, the subgraph is like the one shown ->

Tree

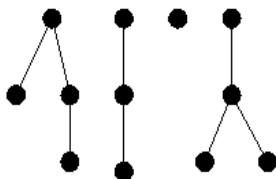


An undirected graph is said to be a *tree* if it contains no cycles and is connected.

A rooted tree

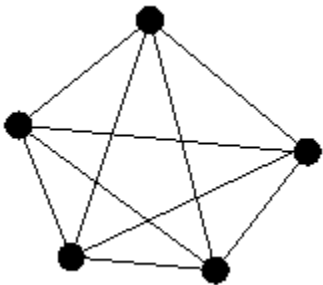
Many trees are what is called *rooted*, where there is a notion of the "top" node, which is called the root. Thus, each node has one *parent*, which is the adjacent node which is closer to the root, and may have any number of *children*, which are the rest of the nodes adjacent to it. The tree above was drawn as a rooted tree.

Forest



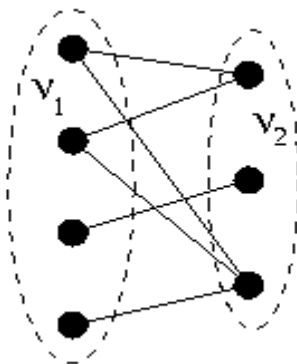
An undirected graph which contains no cycles is called a *forest*. A directed acyclic graph is often referred to as a *dag*.

Complete Graph



A graph is said to be complete if there is an edge between every pair of vertices.

Bipartite Graph



A graph is said to be bipartite if the vertices can be split into two sets V_1 and V_2 such there are no edges between two vertices of V_1 or two vertices of V_2 .

Uninformed Search

Searching is a process of considering possible sequences of actions, first you have to formulate a goal and then use the goal to formulate a problem.

A **problem** consists of four parts: the **initial state**, a set of **operators**, a **goal test** function, and a **path cost** function. The environment of the problem is represented by a

state space. A **path** through the state space from the initial state to a goal state is a **solution**.

In real life most problems are ill-defined, but with some analysis, many problems can fit into the state space model. A single general search algorithm can be used to solve any problem; specific variants of the algorithm embody different strategies. Search algorithms are judged on the basis of **completeness**, **optimality**, **time complexity**, and **space complexity**. Complexity depends on b , the **branching factor** in the state space, and d , the **depth** of the shallowest solution.

This 6 search type below (there are more, but we only show 6 here) classified as uninformed search, this means that the search have no information about the number of steps or the path cost from the current state to the goal - all they can do is distinguish a goal state from a non-goal state. Uninformed search is also sometimes called blind search.

Breadth First Search (BFS)

Breadth-first search expands the shallowest node in the search tree first. It is complete, optimal for unit-cost operators, and has time and space complexity of $O(b^d)$. The space complexity makes it impractical in most cases.

Using BFS strategy, the root node is expanded first, then all the nodes generated by the root node are expanded next, and their successors, and so on. In general, all the nodes at depth d in the search tree are expanded before the nodes at depth $d+1$.

Algorithmically:

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = Dequeue(Q);
        for each v adjacent to u do {
            if (color[v] == WHITE) {
                color[v] = GRAY;
                d[v] = d[u]+1; // compute d[]
                p[v] = u; // build BFS tree
                Enqueue(Q, v);
            }
        }
        color[u] = BLACK;
    }
}
```

BFS runs in $O(V+E)$

Note: BFS can compute $d[v]$ = shortest-path distance from s to v , in terms of minimum number of edges from s to v (un-weighted graph). Its breadth-first tree can be used to represent the shortest-path.

BFS Solution to Popular JAR Problem

```
#include<stdio.h>
#include<conio.h>
#include<values.h>

#define N 105
#define MAX MAXINT

int act[N][N], Q[N*20][3], cost[N][N];
int a, p, b, m, n, fin, na, nb, front, rear;

void init()
{
    front = -1, rear = -1;
    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            cost[i][j] = MAX;
    cost[0][0] = 0;
}

void nQ(int r, int c, int p)
{
    Q[++rear][0] = r, Q[rear][1] = c, Q[rear][2] = p;
}

void dQ(int *r, int *c, int *p)
{
    *r = Q[++front][0], *c = Q[front][1], *p = front;
}

void op(int i)
{
    int currCapA, currCapB;
    if(i==0)
        na = 0, nb = b;
    else if(i==1)
        nb = 0, na = a;
    else if(i==2)
        na = m, nb = b;
    else if(i==3)
        nb = n, na = a;
    else if(i==4)
    {
        if(!a && !b)
```

```

        return;
        currCapB = n - b;
        if(currCapB <= 0)
            return;
        if(a >= currCapB)
            nb = n, na = a, na -= currCapB;
        else
            nb = b, nb += a, na = 0;
    }
    else
    {
        if(!a && !b)
            return;
        currCapA = m - a;
        if(currCapA <= 0)
            return;
        if(b >= currCapA)
            na = m, nb = b, nb -= currCapA;
        else
            nb = 0, na = a, na += b;
    }
}

void bfs()
{
    nQ(0, 0, -1);
    do{

        dQ(&a, &b, &p);
        if(a==fin)
            break;
        for(int i=0; i<6; i++)
        {
            op(i); /* na, nb will be changed for this func
                    according to values of a, b
                    */
            if(cost[na][nb]>cost[a][b]+1)
            {
                cost[na][nb]=cost[a][b]+1;
                act[na][nb] = i;
                nQ(na, nb, p);
            }
        }
    } while (rear!=front);
}

void dfs(int p)
{
    int i = act[na][nb];
    if(p==--1)
        return;
    na = Q[p][0], nb = Q[p][1];
    dfs(Q[p][2]);
    if(i==0)

```

```
        printf("Empty A\n");
    else if(i==1)
        printf("Empty B\n");
    else if(i==2)
        printf("Fill A\n");
    else if(i==3)
        printf("Fill B\n");
    else if(i==4)
        printf("Pour A to B\n");
    else
        printf("Pout B to A\n");
}

void main()
{
    clrscr();
    while(scanf("%d%d%d", &m, &n, &fin)!=EOF)
    {
        printf("\n");
        init();
        bfs();
        dfs(Q[p][2]);
        printf("\n");
    }
}
```

Uniform Cost Search (UCS)

Uniform-cost search expands the least-cost leaf node first. It is complete, and unlike breadth-first search is optimal even when operators have differing costs. Its space and time complexity are the same as for BFS.

BFS finds the shallowest goal state, but this may not always be the least-cost solution for a general path cost function. UCS modifies BFS by always expanding the lowest-cost node on the fringe.

Depth First Search (DFS)

Depth-first search expands the deepest node in the search tree first. It is neither complete nor optimal, and has time complexity of $O(b^m)$ and space complexity of $O(bm)$, where m is the maximum depth. In search trees of large or infinite depth, the time complexity makes this impractical.

DFS always expands one of the nodes at the deepest level of the tree. Only when the search hits a dead end (a non-goal node with no expansion) does the search go back and expand nodes at shallower levels.

Algorithmically:

```

DFS(G) {
  for each vertex u in V
    color[u] = WHITE;
  time = 0; // global variable
  for each vertex u in V

    if (color [u] == WHITE)
      DFS_Visit(u);
}

DFS_Visit(u) {
  color[u] = GRAY;
  time = time + 1; // global variable
  d[u] = time; // compute discovery time d[]
  for each v adjacent to u
    if (color[v] == WHITE) {
      p[v] = u; // build DFS-tree
      DFS_Visit(u);
    }
  color[u] = BLACK;
  time = time + 1; // global variable
  f[u] = time; // compute finishing time f[]
}

```

DFS runs in $O(V+E)$

DFS can be used to classify edges of G :

1. Tree edges: edges in the depth-first forest
2. Back edges: edges (u,v) connecting a vertex u to an ancestor v in a depth-first tree
3. Forward edges: non-tree edges (u,v) connecting a vertex u to a descendant v in a depth-first tree
4. Cross edges: all other edges

An undirected graph is acyclic iff a DFS yields no back edges.

DFS algorithm Implementation

Form a one-element queue consisting of the root node.

Until the queue is empty or the goal has been reached, determine if the first element in the queue is the goal node. If the first element is the goal node, do nothing. If the first element is not the goal node, remove the first element from the queue and add the first element's children, if any, to the **front** of the queue.

If the goal node has been found, announce success, otherwise announce failure.

Note: This implementation differs with BFS in insertion of first element's children, DFS from **FRONT** while BFS from **BACK**. The worst case for DFS is the best case for BFS and vice versa. However, avoid using DFS when the search trees are very large or with infinite maximum depths.

N Queens Problem

Place n queens on an $n \times n$ chess board so that no queen is attacked by another queen.

The most obvious solution to code is to add queens recursively to the board one by one, trying all possible queen placements. It is easy to exploit the fact that there must be exactly one queen in each column: at each step in the recursion, just choose where in the current column to put the queen.

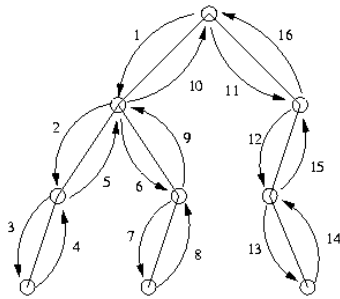
DFS Solution

```
1 search(col)
2   if filled all columns
3     print solution and exit
4   for each row
5     if board(row, col) is not attacked
6       place queen at (row, col)
7       search(col+1)
8       remove queen at (row, col)
```

Calling `search(0)` begins the search. This runs quickly, since there are relatively few choices at each step: once a few queens are on the board, the number of non-attacked squares goes down dramatically.

This is an example of depth first search, because the algorithm iterates down to the bottom of the search tree as quickly as possible: once k queens are placed on the board, boards with even more queens are examined before examining other possible boards with only k queens. This is okay but sometimes it is desirable to find the simplest solutions before trying more complex ones.

Depth first search checks each node in a search tree for some property. The search tree might look like this:



The algorithm searches the tree by going down as far as possible and then backtracking when necessary, making a sort of outline of the tree as the nodes are visited. Pictorially, the tree is traversed in the following manner:

Suppose there are d decisions that must be made. (In this case $d=n$, the number of columns we must fill.) Suppose further that there are C choices for each decision. (In this case $c=n$ also, since any of the rows could potentially be chosen.) Then the entire search will take time proportional to c^d , i.e., an exponential amount of time. This scheme requires little space, though: since it only keeps track of as many decisions as there are to make, it requires only $O(d)$ space.

Knight Cover

Place as few knights as possible on an $n \times n$ chess board so that every square is attacked. A knight is not considered to attack the square on which it sits.

Depth First with Iterative Deepening (DF-ID)

An alternative to breadth first search is iterative deepening. Instead of a single breadth first search, run D depth first searches in succession, each search allowed to go one row deeper than the previous one. That is, the first search is allowed only to explore to row 1, the second to row 2, and so on. This ``simulates" a breadth first search at a cost in time but a savings in space.

```

1 truncated_dfsearch(hnextpos, depth)
2   if board is covered
3     print solution and exit
4   if depth == 0
5     return
6   for i from nextpos to n*n
7     put knight at i
8     search(i+1, depth-1)
9     remove knight at i
10  dfid_search
11  for depth = 0 to max_depth
12    truncated_dfsearch(0, depth)

```


The space complexity of iterative deepening is just the space complexity of depth first search: $O(n)$. The time complexity, on the other hand, is more complex. Each truncated depth first search stopped at depth k takes ck time. Then if d is the maximum number of decisions, depth first iterative deepening takes $c^0 + c^1 + c^2 + \dots + c^d$ time.

Which to Use?

Once you've identified a problem as a search problem, it's important to choose the right type of search. Here are some things to think about.

Search	Time	Space	When to use
DFS	$O(c^k)$	$O(k)$	Must search tree anyway, know the level the answers are on, or you aren't looking for the shallowest number.
BFS	$O(c^d)$	$O(c^d)$	Know answers are very near top of tree, or want shallowest answer.
DFS+ID	$O(c^d)$	$O(d)$	Want to do BFS, don't have enough space, and can spare the time.

d is the depth of the answer, k is the depth searched, $d \leq k$.

Remember the ordering properties of each search. If the program needs to produce a list sorted shortest solution first (in terms of distance from the root node), use breadth first search or iterative deepening. For other orders, depth first search is the right strategy.

If there isn't enough time to search the entire tree, use the algorithm that is more likely to find the answer. If the answer is expected to be in one of the rows of nodes closest to the root, use breadth first search or iterative deepening. Conversely, if the answer is expected to be in the leaves, use the simpler depth first search.

Be sure to keep space constraints in mind. If memory is insufficient to maintain the queue for breadth first search but time is available, use iterative deepening.

Depth Limited Search

Depth-limited search places a limit on how deep a depth-first search can go. If the limit happens to be equal to the depth of shallowest goal state, then time and space complexity are minimized.

DLS stops to go any further when the depth of search is longer than what we have defined.

Iterative Depending Search

Iterative deepening search calls depth-limited search with increasing limits until a goal is found. It is complete and optimal, and has time complexity of $O(b^d)$

IDS is a strategy that sidesteps the issue of choosing the best depth limit by trying all possible depth limits: first depth 0, then depth 1, then depth 2, and so on. In effect, IDS combines the benefits of DFS and BFS.

Bidirectional Search

Bidirectional search can enormously reduce time complexity, but is not always applicable. Its memory requirements may be impractical.

BDS simultaneously search both forward from the initial state and backward from the goal, and stop when the two searches meet in the middle, however search like this is not always possible.

Superprime Rib

A number is called superprime if it is prime and every number obtained by chopping some number of digits from the right side of the decimal expansion is prime. For example, 233 is a superprime, because 233, 23, and 2 are all prime. Print a list of all the superprime numbers of length n , for $n \leq 9$. The number 1 is not a prime.

For this problem, use depth first search, since all the answers are going to be at the n th level (the bottom level) of the search.

Betsy's Tour

A square township has been partitioned into n^2 square plots. The Farm is located in the upper left plot and the Market is located in the lower left plot. Betsy takes a tour of the township going from Farm to Market by walking through every plot exactly once. Write a program that will count how many unique tours Betsy can take in going from Farm to Market for any value of $n \leq 6$.